

DESIGN AND ANALYSIS OF ALGORITHMS — EXAM READY NOTES

SAQ = 2 Marks (write 3–4 lines) | LAQ = 5 Marks (write definition + steps/algorithm + example + complexity)

MODULE – I (Introduction & Divide and Conquer)

SHORT ANSWER QUESTIONS (2 Marks)

1. Define an Algorithm. An algorithm is a **finite sequence of well-defined instructions** to solve a problem, which takes some input, produces an output, and **terminates in finite time**.

2. Characteristics of an Algorithm. (*Memory tip: I-O-D-F-E*)

- **Input** – zero or more inputs
- **Output** – at least one output
- **Definiteness** – each step is clear, unambiguous
- **Finiteness** – terminates after finite steps
- **Effectiveness** – each step is basic enough to be done by hand

3. Time Complexity vs Space Complexity.

- **Time Complexity** – time taken by an algorithm as a function of input size.
- **Space Complexity** – memory required by an algorithm during execution. Both are used to measure and compare algorithm efficiency.

4. Divide and Conquer Technique. A design technique where a problem is **(1) Divided** into smaller subproblems of the same type, **(2) each solved independently (Conquer)**, and **(3) Combined** to get the final solution. Example: Merge Sort, Quick Sort.

5. Multiplications in Strassen's Algorithm.

- Conventional method (2×2 matrices) → **8 multiplications**
- Strassen's method → **7 multiplications + 18 additions/subtractions** (multiplication is costlier than addition, so this saves time).

6. Pivot element in Quick Sort. The pivot is the element chosen to **partition the array**:

- Smaller elements → left of pivot
 - Larger elements → right of pivot
 - After partition, pivot sits in its correct sorted position.
-

LONG ANSWER QUESTIONS (5 Marks)

1. Tower of Hanoi

Definition: A classic recursion puzzle with 3 rods — **Source, Auxiliary, Destination** — and n disks stacked in decreasing size on Source.

Rules:

1. Move one disk at a time.
2. Only the top disk of a rod can be moved.
3. A larger disk can never sit on a smaller disk.

Goal: Move all disks from Source → Destination using Auxiliary, keeping the size order.

Pseudocode:

Algorithm TOH(n , Source, Auxiliary, Destination)

```
{  
  if ( $n > 1$ ) then  
  {  
    TOH( $n-1$ , Source, Destination, Auxiliary)  
    write("Move disk ",  $n$ , " from ", Source, " to ", Destination)  
    TOH( $n-1$ , Auxiliary, Source, Destination)  
  }  
}
```

Example (n = 3):

1. Disk1: A→C 2. Disk2: A→B 3. Disk1: C→B 4. Disk3: A→C
2. Disk1: B→A 6. Disk2: B→C 7. Disk1: A→C

Recurrence Relation: $T(n) = 2T(n-1) + 1 \rightarrow T(n) = 2^n - 1$ moves (for $n=3 \rightarrow 7$ moves)

Complexity: Time = $O(2^n)$, Space = $O(n)$ (recursion stack)

2. Performance Analysis of an Algorithm (Time & Space Complexity)

Definition: Performance = amount of **time and memory** needed to run a program, measured via **analytical (performance analysis)** or **experimental (performance measurement)** methods.

(a) Time Complexity — time as a function of input size n .

(b) Space Complexity — total memory used. $S(P) = C + S_P$

- **C (Fixed part):** program instructions, constants, simple variables — independent of input.
- **S_P (Variable part):** arrays, recursion stack, dynamic memory — grows with input.

Example — Algorithm SUM(A, n):

sum $\leftarrow$ 0

for $i \leftarrow 1$ to n do

 sum $\leftarrow$ sum + $A[i]$

return sum

Statement	Cost	Repetition	Total
sum $\leftarrow$ 0	1	1	1
for condition	1	$n+1$	$n+1$
sum $\leftarrow$ sum + $A[i]$	2	n	$2n$

Statement	Cost	Repetition	Total
return sum	1	1	1

$$T(n) = 3n + 3 \rightarrow T(n) = O(n)$$

$$\text{Space: Array } A(n) + n + i + \text{sum}(3 \text{ words}) \rightarrow S(P) = n+3 \rightarrow S(P) = O(n)$$

3. Binary Search

Definition: A searching technique on a **sorted array** that repeatedly divides the search interval in half.

Pseudocode:

Algorithm BinarySearch(A, n, key)

```
{
    low := 0; high := n-1;

    while (low <= high) do
    {
        mid := (low+high)/2;

        if (A[mid] = key) then return mid;

        if (key < A[mid]) then high := mid-1;

        else low := mid+1;

    }

    return -1;
}
```

Example: A = {10,20,30,40,50,60,70}, key = 50

- Step 1: mid=3, A[3]=40, 50>40 → search right (low=4,high=6)
- Step 2: mid=5, A[5]=60, 50<60 → search left (low=4,high=4)
- Step 3: mid=4, A[4]=50 → **Found at index 4**

Time Complexity: | Case | Complexity | |---|---| | Best | O(1) | | Average | O(log n) | | Worst | O(log n) |

4. Strassen's Matrix Multiplication

Definition: A Divide-and-Conquer method (by Volker Strassen) to multiply two $n \times n$ matrices faster than the $O(n^3)$ conventional method, achieving **$O(n^{2.807})$** .

Method: Split A, B into four $(n/2 \times n/2)$ submatrices each. Compute **7 products**:

$$P = (A_{11}+A_{22})(B_{11}+B_{22}) \quad Q = (A_{21}+A_{22})B_{11}$$

$$R = A_{11}(B_{12}-B_{22}) \quad S = A_{22}(B_{21}-B_{11})$$

$$T = (A_{11}+A_{12})B_{22} \quad U = (A_{21}-A_{11})(B_{11}+B_{12})$$

$$V = (A_{12}-A_{22})(B_{21}+B_{22})$$

Then:

$$C_{11} = P+S-T+V \quad C_{12} = R+T$$

$$C_{21} = Q+S \quad C_{22} = P+R-Q+U$$

Example: A=[1 2;3 4], B=[5 6;7 8] P=65, Q=35, R=-2, S=8, T=24, U=22, V=-30 → **C₁₁=19, C₁₂=22, C₂₁=43, C₂₂=50** → C = [19 22; 43 50]

Recurrence: $T(n) = 7T(n/2) + O(n^2) \rightarrow O(n^{\log_2 7}) = O(n^{2.807})$

5. Divide and Conquer Strategy

Definition: Design technique that breaks a problem into smaller subproblems of the **same type**, solves them independently, and **combines** results.

Basic Steps:

1. **Divide** – split problem into subproblems
2. **Conquer** – solve subproblems recursively (directly if small)
3. **Combine** – merge subproblem solutions

Control Abstraction:

DANDC(P)

```
{  
  
  if SMALL(P) then return S(P);  
  
  else  
  
    {  
  
      divide P into P1, P2, ..., Pk;  
  
      return COMBINE(DANDC(P1), DANDC(P2), ..., DANDC(Pk));  
  
    }  
  
}
```

Recurrence Relation: $T(n) = aT(n/b) + f(n)$

- a = number of subproblems, n/b = size of each subproblem, $f(n)$ = divide/combine cost

Applications: Binary Search, Quick Sort, Merge Sort, Strassen's Matrix Multiplication

6. Merge Sort

Definition: A Divide-and-Conquer sorting algorithm that splits the array, sorts each half, then merges them in sorted order.

Steps: 1) **Divide** recursively into halves 2) **Conquer** — sort each half 3) **Merge** the sorted halves.

Pseudocode:

Algorithm MergeSort(A, low, high)

```

{
    if (low < high) then
    {
        mid := (low+high)/2;

        MergeSort(A, low, mid);

        MergeSort(A, mid+1, high);

        Merge(A, low, mid, high);
    }
}

```

Example: A = [70, 30, 50, 10] Divide → [70,30] [50,10] → [70][30] [50][10] Merge → [30,70] and [10,50] → Final merge → **[10, 30, 50, 70]**

Recurrence: $T(n) = 2T(n/2) + n \rightarrow T(n) = O(n \log n)$ — same for best, average, worst case.

MODULE – II (Backtracking & Disjoint Sets)

SHORT ANSWER QUESTIONS (2 Marks)

1. Hamiltonian Cycle. A cycle in a graph that **visits every vertex exactly once** and returns to the starting vertex. Example: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$.

2. Algorithm for Simple Union Operation.

UNION(i, j)

```

{
    parent[j] ← i;
}

```

```
}
```

Makes root i the parent of root j , merging the two sets into one.

3. State-space tree in backtracking. A tree representation of all possible states/solutions explored while solving a problem using backtracking; invalid branches are pruned.

4. Find operation in disjoint-set. The **Find** operation returns the **root (representative)** of the set that contains a given element.

5. How is a Heap implemented? A heap is implemented as an **array representing a complete binary tree**; parent/child indices are computed arithmetically, giving efficient insertion and deletion.

6. Sum of Subsets Problem. Given a set of positive integers and a target sum d , find all subsets whose elements add up exactly to d , using **backtracking**.

LONG ANSWER QUESTIONS (5 Marks)

1. Disjoint Set Operations — Union & Find

Definition: Disjoint sets have **no common elements** (e.g., $S1=\{1,7,8,9\}$, $S2=\{2,5,10\}$). Two operations: **Union** (merge two sets) and **Find** (identify which set an element belongs to).

Simple Union: `SimpleUnion(i, j) { P[j] = i; }` — i (root of set 1) becomes parent of j (root of set 2).

Simple Find:

Find(i)

```
{
```

```
    while (P[i] > 0) i = P[i];
```

```
    return i;
```

```
}
```

Follows parent links until a node with parent **-1** (root) is found.

Example: Elements 1–6, all $P[i] = -1$ initially. Union(1,2), Union(3,4), Union(5,6), Union(1,3), Union(1,5) → Final tree: 1 is root of {1,2,3,4,5,6} **Find(4):** $4 \rightarrow P[4] = 3 \rightarrow 3 \rightarrow P[3] = 1 \rightarrow P[1] = -1 \rightarrow \text{root} = 1$

Time Complexity: Simple Union = $O(1)$; Simple Find = $O(n)$ worst case.

2. N-Queens Problem using Backtracking (4-Queens)

Definition: Place N queens on an $N \times N$ board so that **no two queens attack** each other (no same row, column, or diagonal).

Method: Place queens **row by row**; if a position is unsafe, **backtrack** to the previous row and try the next column.

4-Queens trace:

- Q1 → column 1 → unsafe combos with Q2, Q3 force backtracking
- After backtracking through several placements, valid solution found: **$x_1=2, x_2=4, x_3=1, x_4=3$** i.e., Queen1→col2, Queen2→col4, Queen3→col1, Queen4→col3

Time Complexity: Worst case **$O(N!)$** , Space = **$O(N)$**

3. General Method of Backtracking

Definition: A **trial-and-error** problem-solving technique that builds a solution incrementally and **abandons (backtracks)** a path as soon as it determines the path cannot lead to a valid solution.

Terminology:

- **Root node** – initial state
- **Live node** – generated but not yet explored
- **E-node** – node currently being expanded
- **Dead node** – cannot lead to a solution (discarded)
- **Answer node** – represents a valid complete solution

Algorithm:

Backtrack(k)

```
{  
  
  if solution found: print solution  
  
  else  
  
    for each candidate x  
  
      if x is promising  
  
        {  
  
          include x  
  
          Backtrack(k+1)  
  
          remove x      // backtrack step  
  
        }  
  
}
```

Steps: Start at root → generate candidate → check if promising → continue if yes / backtrack if no → repeat till all explored.

Applications: N-Queens, Graph Coloring, Hamiltonian Cycle, Sum of Subsets.

4. State Space Tree for Sum of Subsets (N=4, M=50, W={10,20,30,40})

Definition: Find subsets of W whose sum = target M = 50, using backtracking (1 = include element, 0 = exclude).

Algorithm:

1. If sum = d → print subset, return
2. If sum > d or i = n → return (dead node)
3. Include S[i]: SumOfSubsets(i+1, sum+S[i])

4. Exclude $S[i]$: SumOfSubsets($i+1$, sum)

Trace (key steps): | Subset | Sum | Result | |---|---|---| | {10,20,30} | 60 | Backtrack | | {10,20,40} | 70 | Backtrack | | {10,30,40} | 80 | Backtrack | | **{10,40}** | **50** | **Solution** | | **{20,30}** | **50** | **Solution** |

Answer: Two solutions $\rightarrow$ **{10,40}** and **{20,30}**

5. Graph Coloring Problem using Backtracking

Definition (m-colorability): Given graph G and integer m , determine if vertices can be colored using only **m colors** so that **no two adjacent vertices share a color**.

Algorithm:

```
mColoring(k)
{
    while(true)
    {
        NextValue(k)

        if  $x[k] == 0$  return      // no color available  $\rightarrow$  backtrack

        if  $k == n$  print( $x[1..n]$ )  // solution found

        else mColoring( $k+1$ )
    }
}
```

NextValue(k): try next color for vertex $k \rightarrow$ check against all adjacent vertices $\rightarrow$ accept if no conflict, else try next color.

Example: Cycle graph A-B-C-D-A (4 vertices). Using colors {Red, Green, Blue}:

- A = Red
- B = Green (adjacent to A)
- C = Red (adjacent only to B)
- D = Green (adjacent to A & C, both used Red)

Final coloring: A=Red, B=Green, C=Red, D=Green — **valid, no two adjacent vertices share a color.**

Time Complexity: Worst case $O(m^n)$; Space = $O(n)$

6. Hamiltonian Cycle using Backtracking

Definition: A cycle that **visits every vertex exactly once** and returns to the start vertex.

Conditions to add a vertex to the path:

1. Must be connected (adjacent) to the previous vertex
2. Must not already be used
3. If it's the last vertex, it must connect back to the start vertex

Algorithm:

Hamiltonian(k)

{

if all vertices included: print cycle

else

for every vertex v

if v is valid: add v to path; Hamiltonian(k+1); remove v (backtrack)

}

Example graph: vertices 1,2,3,4,5 (1-2, 2-5, 5-4, 4-3, 3-1 edges) Path built: 1 → 2 → 5 → 4 → 3 → check 3 connects to 1? Yes ✓ **Hamiltonian Cycle = 1→2→5→4→3→1**

Time Complexity: Worst case $O(n!)$; Space = $O(n)$

UNIT – III (Dynamic Programming)

SHORT ANSWER QUESTIONS (2 Marks)

1. Dynamic Programming (with example). DP is an algorithmic technique that solves optimization problems by breaking them into **overlapping subproblems**, solving each **only once**, storing the result, and reusing it. **Example:** Fibonacci — $F(n) = F(n-1) + F(n-2)$; DP avoids recomputing $F(3)$, $F(2)$ repeatedly.

2. Key properties of Dynamic Programming.

1. **Optimal Substructure** – the optimal solution of the problem is built from optimal solutions of its subproblems.
2. **Overlapping Subproblems** – the same subproblems recur multiple times, so results are stored and reused.

3. Applications of Dynamic Programming.

- Optimal Binary Search Tree
- 0/1 Knapsack Problem
- All Pairs Shortest Path (Floyd–Warshall)
- Traveling Salesperson Problem
- Reliability Design

LONG ANSWER QUESTIONS (5 Marks)

1. Divide & Conquer (D&C) vs Dynamic Programming (DP)

D&C	DP
Divides into independent subproblems	Divides into overlapping subproblems
Steps: Divide → Conquer → Combine	Steps: Characterize → Recurrence → Compute → Construct solution

D&C	DP
Uses recursion (top-down)	Mostly bottom-up (tabulation) or top-down with memoization
Solves same subproblem repeatedly	Solves each subproblem once , stores result
No storage of intermediate results	Stores intermediate results (table)
May do redundant work	Eliminates redundant computation
Examples: Merge Sort, Quick Sort, Binary Search, Strassen's	Examples: 0/1 Knapsack, OBST, Floyd–Warshall, TSP

2. Dynamic Programming — Definition, Elements & Control Abstraction

Definition: DP solves optimization/decision problems by breaking them into **overlapping subproblems**, solving each once, storing results, and combining them for the optimal answer.

Elements of DP:

- **Memoization** – top-down; store solved subproblem results, reuse when needed.
- **Tabulation** – bottom-up; compute subproblem solutions iteratively in a table.
- **Principle of Optimality** – whatever the initial state/decision, the remaining decisions must form an optimal sequence w.r.t. the state resulting from the first decision.
- **Optimal Substructure** – optimal solution built from optimal solutions of subproblems.
- **Overlapping Subproblems** – same subproblems recur, causing exponential blow-up if not stored.

Four Steps to design a DP algorithm:

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value bottom-up.
4. Construct the optimal solution from computed information.

Control Abstraction:

Algorithm DYNAMIC_PROGRAMMING(P)

{

```

    if solved(P) then return lookup(P)

    else { Ans  $\leftarrow$  SOLVE(P); store(P, Ans) }

}

Function SOLVE(P)

{

    if sufficiently small(P) then solution(P)

    else

    {

        divide P into P1, P2, ..., Pn

        Ans1  $\leftarrow$  DYNAMIC_PROGRAMMING(P1)

        ...

        Ansn  $\leftarrow$  DYNAMIC_PROGRAMMING(Pn)

        return combine(Ans1, ..., Ansn)

    }

}

```

3. 0/1 Knapsack Problem — Worked Example

Given: $n = 4$ items, weights $w = (1, 2, 2, 3)$, profits $p = (1, 3, 4, 5)$, knapsack capacity **$W = 4$** .

Note: solved below using the standard DP table method as the question requires.

DP Table $K[i][w]$ — rows = items considered (0 to 4), columns = capacity (0 to 4):

$i \setminus w$	0	1	2	3	4
0 (none)	0	0	0	0	0

$i \setminus w$	0	1	2	3	4
1 ($w=1, p=1$)	0	1	1	1	1
2 ($w=2, p=3$)	0	1	3	4	4
3 ($w=2, p=4$)	0	1	4	5	7
4 ($w=3, p=5$)	0	1	4	5	7

Recurrence used: $K[i][w] = \max(K[i-1][w], p_i + K[i-1][w-w_i]) \quad \text{if } w_i \leq w$
 $K[i][w] = K[i-1][w] \quad \text{if } w_i > w$

Maximum Profit = $K[4][4] = 7$

Trace back (which items selected):

- $K[4][4]=7 = K[3][4]=7 \rightarrow$ **Item 4 NOT selected**
- $K[3][4]=7 \neq K[2][4]=4 \rightarrow$ **Item 3 selected** $\rightarrow$ remaining capacity = $4-2 = 2$
- $K[2][2]=3 \neq K[1][2]=1 \rightarrow$ **Item 2 selected** $\rightarrow$ remaining capacity = $2-2 = 0$
- $K[1][0]=0 = K[0][0]=0 \rightarrow$ **Item 1 NOT selected**

Final Answer:

- **Selected Items:** Item 2 and Item 3
- **Total Weight:** $2 + 2 = 4$
- **Total Profit:** $3 + 4 = 7$
- **Decision Vector $(x_1, x_2, x_3, x_4) = (0, 1, 1, 0)$**

Quick Revision Table — Time Complexities

Algorithm	Best	Average	Worst
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Strassen's Matrix Mult.	—	$O(n^{2.807})$	$O(n^{2.807})$
Tower of Hanoi	—	$O(2^n)$	$O(2^n)$

Algorithm	Best	Average	Worst
N-Queens (backtracking)	—	—	$O(N!)$
Graph Coloring (backtracking)	—	—	$O(m^n)$
Hamiltonian Cycle (backtracking)	—	—	$O(n!)$
0/1 Knapsack (DP)	$O(nW)$	$O(nW)$	$O(nW)$